

Names: Rhilo Sotto, John Hayali, Eren Ugur

Report for Programming Assignment #2

Explanation of Implementation of Different Algorithms

First come first serve (FCFS)

We made changes to the sample code to have the processes execute until completion rather than using a quantum value. A SIGCONT was sent sequentially to each process to resume and the main program waited for the child to finish using `waitpid(pid, NULL, 0)`. This gives the pure FCFS scheduling behavior in which there is no preemption and every process executes until completion when it begins. We made the assumption that all processes arrived at time 0 and no additional processes would emerge in the process of execution. Response time was measured by recording timestamps at the start and completion of each process using the `clock_gettime()` function. Our control group was the FCFS implementation, which illustrated the convoy effect of short jobs trailing behind the long jobs.

Shortest job first (SJF)

In order to apply the SJF algorithm, the following process was chosen according to the minimal workload, and we assumed that the less the work, the shorter the job. After creating and assigning workloads to all four processes, each was paused with SIGSTOP. The scheduler then compared workloads and mapped the tasks in order of smallest workloads. In case of a tie, the earlier-created process ran first. Because SJF is non-preemptive, each job ran to completion before the next began. An array tracked process status, and response time was measured using `clock_gettime()` between process start and finish. The results confirmed that SJF reduced the average turnaround time by prioritizing short jobs, although longer tasks could starve when many shorter ones existed.

Multi-level feedback queue (MLFQ)

We used the MLFQ algorithm by making the RR design with 2 queues. The first is RR design with a short quantum, and the second is the FCFS design, which is assigned to any process that had not moved to completion in the first queue. All the process began in Queue 1 and used `usleep(QUANTUM1)` to execute one quantum. At the end of the time slice, we verified its status with `waitpid(WNOHANG)`. If the process had not completed within its quantum, we suspended it with SIGSTOP and demoted it to Queue 2. After all processes were completed in Queue 1, those that were not completed were resumed in Queue 2 and run to completion on FCFS logic. We have assumed that all the processes are received at the same time, and the length of the quantum is equal to all the jobs. An array was used to keep track of task running status. As in the other algorithms, response time was measured from the start to the finish of each process using `clock_gettime()`. This method enabled the completion of shorter jobs in the early stages and provided lasting jobs with the remaining CPU time in later stages, giving MLFQ a balance between fairness and efficiency.

Results of Experiments

[\[Link to Spreadsheet\]](#)

The specifications of the CPU that was a part of the system that ran the tests are “Intel(R) Core(TM) i7-10510U CPU @ 1.80GHz” (4 Cores and 8 Threads).

Experiment 1

For this experiment, we performed 100 tests for each quantum value for Round Robin. Moreover, the time quantum values that we tested in this experiment were 100, 3000, 5000, 6000, 7000, 10000, 15000, 25000, 50000, 100000, and 2500000. The reason why we chose these values is because we aimed to find the optimal value through a searching method that is similar to binary search. Specifically, we tested a small value ($Q = 100$) and a large one ($Q = 2500000$). Then, we chose a value that is roughly in between these two values (aka $Q = 100000$). Since the average response time of $Q = 100000$ was less than the average response time of $Q = 2500000$, we chose a value in between $Q = 100$ and $Q = 100000$. We repeated this process until we identified an optimal value. The table below displays the statistics obtained from our experiment:

	Average T_r	Standard Deviation of T_r	Average # of Context Switches	Average Context-Switch Time
$Q = 100$	1.0387 s	0.0374 s	16307.68	0.017411 s
$Q = 3000$	0.9420 s	0.0407 s	746.44	0.002056 s
$Q = 5000$	0.9175 s	0.0522 s	437.18	0.001207 s
$Q = 6000$	0.9270 s	0.0800 s	367.24	0.001012 s
$Q = 7000$	0.9234 s	0.0597 s	314.64	0.000878 s
$Q = 10000$	0.9264 s	0.0480 s	221.55	0.000622 s
$Q = 15000$	0.9922 s	0.0420 s	153.77	0.000453 s
$Q = 25000$	0.9391 s	0.0229 s	87.81	0.000245 s
$Q = 50000$	1.0173 s	0.0291 s	44.59	0.000145 s
$Q = 100000$	1.2402 s	0.0156 s	24.53	0.000091 s
$Q = 2500000$	6.2519 s	0.0005 s	3	0.000023 s

Among these values, the optimal time quantum is $Q = 5000$ because it has a lower average response time compared to the other values tested.

There are different average response times for different quantum values because of context switches and unnecessary waiting. For example, if Q is much smaller than the optimal value, then the system will experience more context switches. On the other hand, if Q is much larger than the optimal value, then the system will experience more unnecessary waiting as a result of tasks that have completed their workload sooner than their quantum finished.

Furthermore, it would help to give a specific quantum value for each process because the workload sizes differ. For instance, a task with a smaller workload can be given a smaller time quantum compared to a task with a larger workload. This can help prevent the main program from unnecessarily waiting if a process is already completed. By doing this, it is possible to achieve the fastest response times for each task.

In addition, we estimated the context switch overhead results by taking timestamps before SIGSTOP and after SIGCONT and accumulating the difference. Context switches occur when a task is paused/terminated and another task begins/resumed (the first task running and the final task terminating are not considered context switches). It is also important to note that measuring context switching overhead is additional overhead as well.

Experiment 2

For this experiment, we performed 100 tests for each quantum value for Multi-Level Feedback Queue. Moreover, the time quantum values that we tested in this experiment were 1000, 20000, 23000, 25000, 27000, 30000, 35000, 50000, 75000, 100000, and 2500000. We chose these values by utilizing the same “binary search” method described previously. The table below displays the statistics obtained from our experiment:

	Average T_r	Standard Deviation of T_r	Average # of Context Switches	Average Context-Switch Time
$Q = 1000$	2.1236 s	0.0579 s	7	0.000021 s
$Q = 20000$	1.9943 s	0.2845 s	6.7	0.000017 s
$Q = 23000$	1.6388 s	0.0724 s	6.01	0.000017 s
$Q = 25000$	1.5909 s	0.0626 s	6	0.000024 s
$Q = 27000$	1.5935 s	0.0639 s	6	0.000017 s
$Q = 30000$	1.5915 s	0.0657 s	6	0.000017 s
$Q = 35000$	1.5962 s	0.0623 s	6	0.000017 s
$Q = 50000$	1.6362 s	0.0533 s	6	0.000026 s
$Q = 75000$	1.6850 s	0.0563 s	6	0.000017 s

Q = 100000	1.7552 s	0.0530 s	6	0.000028 s
Q = 2500000	6.2520 s	0.0005 s	3	0.000023 s

Among these values, the optimal time quantum is $Q = 25000$ because it has a lower average response time compared to the other values tested.

In the MLFQ scheduler, a very small quantum will result in practically all jobs being pushed to the lower queue, therefore making the system act as FCFS-only once the first round is over. An extremely large quantum, on the other hand, is the one that places most of the jobs in the upper Round Robin queue, and therefore becomes RR-only. The two extremes decrease the benefit of the two-level design; optimal behaviour is achieved when the quantum is balanced so that short jobs can complete in Queue 1 and longer jobs can restart in Queue 2 fairly.

We used the same methodology as Round Robin to estimate the context switch overhead. The amount of context switches is relatively constant compared to Round Robin, since the RR queue is only one layer and it demotes any unfinished tasks. It is notable that there are certain thresholds for the time quantum that reduce the amount of context switches, which in this case is caused by the shorter tasks completing within the time quantum. Although we don't observe the exact point where each task meets the breakpoint, the optimal time quantum meets the threshold for task 4 to run to completion within its quantum.

Experiment 3 (Case 3a)

For this experiment, we performed 100 tests for each implementation: RR, SJF, FCFS, and MLFQ. The table below displays the statistics obtained from our experiment:

	Average T_r	Standard Deviation of T_r	Average # of Context Switches	Average Context-Switch Time
Round Robin (Q = 5000)	0.9197 s	0.0631 s	438.5	0.001226 s
SJF	0.8843 s	0.0082 s	3	0.000006 s
FCFS	2.4409 s	0.0384 s	3	0.000007 s
MLFQ (Q = 25000)	1.6412 s	0.0231 s	6	0.000017 s

According to this table, SJF is the best in terms of average response time. This outcome supports the theoretical information we saw in class.

Experiment 3 (Case 3b)

For this experiment, we performed 100 tests for each implementation: RR, SJF, FCFS, and MLFQ. The table below displays the statistics obtained from our experiment:

	Average T_r	Standard Deviation of T_r	Average # of Context Switches	Average Context-Switch Time
Round Robin (Q = 5000)	7.2583 s	1.3692 s	1427.96	0.004386 s
SJF	4.9786 s	0.0436 s	3	0.000010 s
FCFS	4.9593 s	0.0512 s	3	0.000010 s
MLFQ (Q = 25000)	4.4960 s	0.0430 s	7	0.000027 s

According to this table, MLFQ seems to be the best in terms of average response time. However, according to the theoretical information we saw in class, SJF should still be the best (like in Case 3a). For this experiment in particular where every task has the same workload, FCFS and SJF should have the similar response times (though SJF has the additional sorting algorithm overhead cost). This is likely caused by an issue relating to the system that ran the experiment. For instance, when we ran the same experiment on another system with a “Intel® Core™ Ultra 7 256V × 8” processor, the average response times of RR, MLFQ, FCFS, and SJF were 11.3075 s, 1.8174 s, 1.7320 s, and 1.7307, respectively.

It would make sense to use different time quantum values for Round Robin and MLFQ for this experiment since the optimal response time schedule SJF devolves to FCFS, so theoretically FCFS is the best schedule that can be used in this instance.